

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Constraint-Based Problem Solving (High)
Assessment Tool Weightage:40%

Dr

QUESTIONS		
Q.No	Question	Bloom's Level
1	Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.	BL4 – Analyze
2	Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.	BL4 – Analyze
3	Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with Prepared Statement.	BL3 – Apply
4	Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.	BL4 – Analyze
5	Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.	BL3 – Apply
6	Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.	BL4 – Analyze

7	Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.	BL3 – Apply
8	Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.	BL3 – Apply
9	Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.	BL4 – Analyze
10	Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.	BL4 – Analyze

Code of Conduct:

*I **R. Pranathi(192525076)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: 

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints

		satisfying all constraints			
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application ; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.

ANS:

```
mysql> CREATE DATABASE college_db;
Query OK, 1 row affected (0.02 sec)

mysql> USE college_db;
Database changed
mysql> CREATE TABLE STUDENT ( Student_ID INT PRIMARY KEY, Name VARCHAR(50), Dept_ID INT );
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE DEPARTMENT ( Dept_ID INT PRIMARY KEY, Dept_Name VARCHAR(50) );
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE MARKS ( Student_ID INT, Subject VARCHAR(50), Mark INT );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO DEPARTMENT VALUES (1, 'CSE'), (2, 'ECE'), (3, 'IT');
Query OK, 3 rows affected (0.02 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO STUDENT VALUES (101, 'Anjali', 1), (102, 'Rahul', 2), (103, 'Pranathi', 1), (104, 'Kiran', 3);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> INSERT INTO MARKS VALUES (101, 'DBMS', 90), (102, 'DBMS', 75), (103, 'DBMS', 85), (104, 'DBMS', 70);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT S.Name, D.Dept_Name, M.Mark FROM STUDENT S JOIN DEPARTMENT D ON S.Dept_ID = D.Dept_ID JOIN MARKS M ON S.Student_ID = M.Student_ID WHERE D.Dept_Name = 'CSE' AND M.Mark > 80;
+-----+-----+-----+
| Name | Dept_Name | Mark |
+-----+-----+-----+
| Anjali | CSE | 90 |
| Pranathi | CSE | 85 |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

Draw the Initial Query Execution Plan

The original query can be represented as:

```

SELECT
  Dept_Name = 'CSE'
  AND Mark > 80
  |
  JOIN
  /      \
JOIN      MARKS
/        \
STUDENT   DEPARTMENT

```

This is not the most efficient plan because the selection conditions are applied after the joins.

Apply Heuristic Rule 1 – Push Selection Down

The conditions are:

D.Dept_Name = 'CSE'

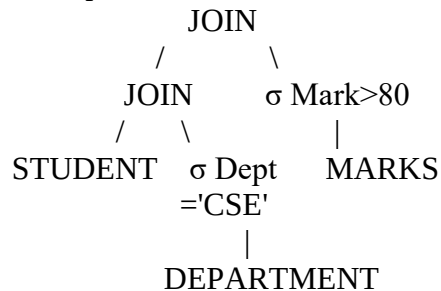
M.Mark > 80

Instead of waiting until after the joins, apply them directly to the tables.

DEPARTMENT
|
Dept_Name='CSE'

MARKS
|
Mark>80

The optimized tree becomes:



Apply Heuristic Rule 2 – Push Projection Down

Only the required columns should be retained.

From STUDENT, we need:

Student_ID
Name
Dept_ID

From DEPARTMENT, we need:

Dept_ID
Dept_Name

From MARKS, we need:

Student_ID
Mark

Therefore:

π Student_ID, Name, Dept_ID (STUDENT)

π Dept_ID, Dept_Name
 σ Dept_Name='CSE' (DEPARTMENT)

π Student_ID, Mark
 σ Mark>80 (MARKS)

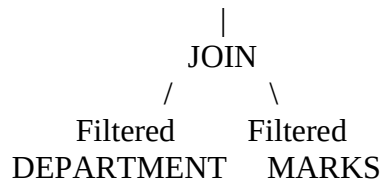
This reduces the amount of data transferred between operations.

Apply Heuristic Rule 3 – Perform Smaller Operations First

DEPARTMENT $\rightarrow$ Dept_Name = 'CSE'
MARKS $\rightarrow$ Mark > 80

Then perform the joins.

STUDENT



Final Optimized Query

```

SELECT S.Name, D.Dept_Name, M.Mark
FROM STUDENT S
JOIN
(
  SELECT Dept_ID, Dept_Name
  FROM DEPARTMENT
  WHERE Dept_Name = 'CSE'
) D
ON S.Dept_ID = D.Dept_ID
JOIN
(
  SELECT Student_ID, Mark
  FROM MARKS
  WHERE Mark > 80
) M
ON S.Student_ID = M.Student_ID;

```

Check the Execution Plan in MySQL

Use:

EXPLAIN

```

SELECT S.Name, D.Dept_Name, M.Mark
FROM STUDENT S
JOIN DEPARTMENT D
  ON S.Dept_ID = D.Dept_ID
JOIN MARKS M
  ON S.Student_ID = M.Student_ID
WHERE D.Dept_Name = 'CSE'
AND M.Mark > 80;

```

Result

The heuristic optimization follows these rules:

Original Query



Push Selection Down



Push Projection Down



Reduce Intermediate Results



Perform Smaller Joins First



Optimized Query



Final Result

Conclusion

Heuristic optimization improves query performance by reducing the number of tuples and attributes before performing expensive join operations. This reduces memory usage, disk I/O, and query execution time.

2. Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.

ANS:

Given Information

Consider two relations:

- Relation **R** = 1,000 tuples
- Relation **S** = 5,000 tuples
- Each page stores **10 tuples**
- Available buffer pages = **11**

Given Table

Relation	Number of Tuples	Tuples per Page	Number of Pages
R	1,000	10	100
S	5,000	10	500

Step 2: Calculate Number of Pages

For Relation R

$$\begin{aligned}B(R) &= \text{Number of tuples} / \text{Tuples per page} \\ &= 1000 / 10 \\ &= 100 \text{ pages}\end{aligned}$$

For Relation S

$$\begin{aligned}B(S) &= \text{Number of tuples} / \text{Tuples per page} \\ &= 5000 / 10 \\ &= 500 \text{ pages}\end{aligned}$$

Therefore:

$$R = 100 \text{ pages}$$

$$S = 500 \text{ pages}$$

Relation **R** is smaller.

Calculate Nested-Loop Join Cost

For Block Nested-Loop Join:

$$\text{Cost} = B(R) + \text{ceil}(B(R)/(M-2)) \times B(S)$$

Where:

- $B(R) = 100$
- $B(S) = 500$
- $M = 11$ buffer pages

Substitute the values:

$$\begin{aligned}\text{Cost} &= 100 + \text{ceil}(100/(11-2)) \times 500 \\ &= 100 + \text{ceil}(100/9) \times 500 \\ &= 100 + 12 \times 500 \\ &= 100 + 6000 \\ &= 6100 \text{ I/O}\end{aligned}$$

Nested-Loop Join Cost

$$= 6100 \text{ I/O operations}$$

Calculate Sort-Merge Join Cost

For external sorting, the approximate cost is:

$$\text{Sort Cost} = 2 \times B \times \text{Number of passes}$$

For the given buffer size, we can estimate:

Sorting R

R contains 100 pages.

$$\text{Sort}(R) \approx 400 \text{ I/O}$$

Sorting S

S contains 500 pages.

$$\text{Sort}(S) \approx 3000 \text{ I/O}$$

After sorting, the final merge requires:

$$\begin{aligned} & B(R) + B(S) \\ &= 100 + 500 \\ &= 600 \text{ I/O} \end{aligned}$$

Therefore:

$$\begin{aligned} \text{Total Sort-Merge Cost} \\ &= 400 + 3000 + 600 \\ &= 4000 \text{ I/O} \end{aligned}$$

Sort-Merge Join Cost

$$\approx 4000 \text{ I/O operations}$$

The exact sort-merge cost can vary depending on the external sorting assumptions. State the assumptions used in your calculation.

Calculate Hash Join Cost

For a **two-pass Hash Join**, the approximate cost is:

$$\text{Cost} = 3 \times (B(R) + B(S))$$

Substitute the values:

$$\begin{aligned} \text{Cost} &= 3 \times (100 + 500) \\ &= 3 \times 600 \\ &= 1800 \text{ I/O} \end{aligned}$$

Hash Join Cost

$$= 1800 \text{ I/O operations}$$

Compare the Costs

Join Algorithm	Estimated Cost
Nested-Loop Join	6100 I/O
Sort-Merge Join	≈ 4000 I/O
Hash Join	1800 I/O

Select the Best Join

The optimizer chooses the algorithm with the **lowest cost**.

Nested-Loop Join $\rightarrow$ 6100 I/O

Sort-Merge Join $\rightarrow$ 4000 I/O

Hash Join $\rightarrow$ 1800 I/O

Therefore:

Hash Join is selected.

Because:

$$1800 < 4000 < 6100$$

Why Hash Join is Selected

Hash Join is the best choice because:

1. It has the lowest estimated I/O cost.

2. It is efficient for equality-based joins.
3. It avoids repeatedly scanning the larger relation.
4. It is faster than Nested-Loop Join for these relation sizes.
5. It requires fewer disk I/O operations.

Final Answer

Given:

R = 1000 tuples = 100 pages

S = 5000 tuples = 500 pages

Buffer pages = 11

Nested-Loop Join = 6100 I/O

Sort-Merge Join $\approx$ 4000 I/O

Hash Join = 1800 I/O

Therefore,

Hash Join has the lowest cost



Hash Join is selected

Result

The cost-based optimizer selects Hash Join because its estimated cost of 1800 I/O operations is lower than Nested-Loop Join (6100 I/O) and Sort-Merge Join (approximately 4000 I/O).

3. Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with Prepared Statement.

ANS:

```
mysql> CREATE DATABASE college_db;
Query OK, 1 row affected (0.09 sec)

mysql> USE college_db;
Database changed
mysql> CREATE TABLE student ( id INT PRIMARY KEY, name VARCHAR(50), email VARCHAR(100), marks INT );
Query OK, 0 rows affected (0.10 sec)

mysql> INSERT INTO student VALUES(101, 'Anjali', 'anjali@gmail.com', 85),(102, 'Rahul', 'rahul@gmail.com', 75),(103, 'Pranathi', 'pranathi@gmail.com', 90),(104, 'Kiran', 'kiran@gmail.com', 65);
Query OK, 4 rows affected (0.06 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM student;
+----+-----+-----+-----+
| id | name  | email          | marks |
+----+-----+-----+-----+
| 101 | Anjali | anjali@gmail.com | 85    |
| 102 | Rahul  | rahul@gmail.com  | 75    |
| 103 | Pranathi | pranathi@gmail.com | 90    |
| 104 | Kiran  | kiran@gmail.com  | 65    |
+----+-----+-----+-----+
4 rows in set (0.04 sec)
```

Why PreparedStatement is Used

Prepared Statement is better than directly joining user input into SQL because:

1. Prevents SQL Injection
2. Supports parameterized queries
3. Improves security
4. Makes queries easier to reuse
5. Provides better handling of different data types

Execution Flow

Java Program

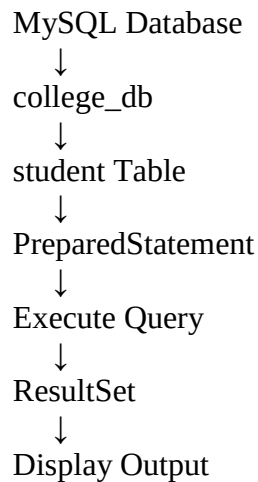


DriverManager



MySQL JDBC Driver





Result

Thus, JDBC connectivity between Java and MySQL was successfully implemented. A parameterized SQL query was executed using PreparedStatement, and the required student records were retrieved successfully.

4. Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.

ANS:

Consider the Given Schedule

Let the four transactions be T1, T2, T3, and T4.

Consider the following schedule:

Step	Transaction	Operation
1	T1	R1(A)
2	T2	R2(A)
3	T1	W1(A)
4	T3	R3(B)
5	T2	W2(A)
6	T4	R4(B)
7	T3	W3(B)
8	T4	W4(B)

Here:

- R = Read
- W = Write
- A and B = Data items

Identify Conflicting Operations

Two operations conflict when:

1. They belong to different transactions.
2. They access the same data item.
3. At least one operation is a write.

Conflicts on A

We have:

T1: R1(A)

T1: W1(A)

T2: R2(A)

T2: W2(A)

Important conflicts: $R1(A) \rightarrow W2(A)$ $W1(A) \rightarrow R2(A)$ $W1(A) \rightarrow W2(A)$

Therefore:

 $T1 \rightarrow T2$ **Conflicts on B**

We have:

 $T3: R3(B)$ $T3: W3(B)$ $T4: R4(B)$ $T4: W4(B)$ **Important conflicts:** $R3(B) \rightarrow W4(B)$ $W3(B) \rightarrow R4(B)$ $W3(B) \rightarrow W4(B)$

Therefore:

 $T3 \rightarrow T4$ **Create the Precedence Graph**

Create one node for each transaction:

 $T1 \quad T2 \quad T3 \quad T4$

Now add edges according to the conflicts:

 $T1 \longrightarrow T2$ $T3 \longrightarrow T4$

Precedence Graph

 $T1 \longrightarrow T2$ $T3 \longrightarrow T4$ **Check for a Cycle**

Now check whether the graph contains a cycle.

The graph has:

 $T1 \rightarrow T2$ $T3 \rightarrow T4$

There is no path that returns to the same transaction.

Therefore:

No Cycle

↓

Acyclic Precedence Graph

↓

Conflict Serializable

Possible Serial Order

Since there is no cycle, an equivalent serial order can be obtained using topological ordering.

One possible order is:

 $T1 \rightarrow T2 \rightarrow T3 \rightarrow T4$

Another valid order could be:

$T3 \rightarrow T4 \rightarrow T1 \rightarrow T2$
Both satisfy the precedence constraints.

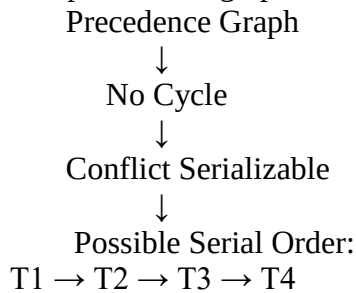
Result

Decision

The given schedule is conflict-serializable.

Reason

The precedence graph contains no cycle.



Conclusion

Therefore, the schedule of the four concurrent transactions is conflict-serializable because its precedence graph is acyclic. The schedule is equivalent to a serial execution such as $T1 \rightarrow T2 \rightarrow T3 \rightarrow T4$.

5. Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.

ANS:

Understand Basic 2PL

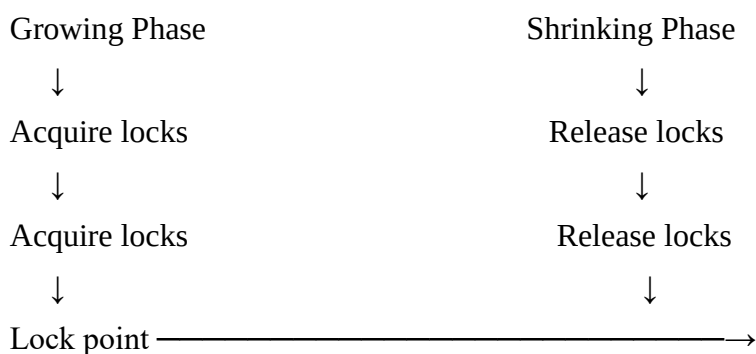
Two-Phase Locking divides a transaction into two phases:

Phase 1 – Growing Phase

- Transaction can acquire locks.
- Transaction cannot release locks.

Phase 2 – Shrinking Phase

- Transaction can release locks.
- Transaction cannot acquire any new locks.



Given Transaction Set

Consider two transactions:

Transaction T1

T1:

Read(A)

Write(A)

Read(B)

Write(B)

Transaction T2

T2:

Read(B)

Write(B)

Read(A)

Write(A)

Both transactions need A and B, but they request them in different orders.

Apply Locks

We use:

- S-lock = Shared lock → used for reading
- X-lock = Exclusive lock → used for writing

Since both transactions write A and B, they need X-locks.

T1

T1 → X-lock(A)

T1 → Read(A)

T1 → Write(A)

T2

T2 → X-lock(B)

T2 → Read(B)

T2 → Write(B)

At this point:

T1 holds X-lock(A)

T2 holds X-lock(B)

Deadlock Occurs

Now T1 requests B:

T1 → Request X-lock(B)

But B is already locked by T2.

Therefore:

$T1 \rightarrow \text{WAIT for } T2$

Now $T2$ requests A :

$T2 \rightarrow \text{Request X-lock}(A)$

But A is already locked by $T1$.

Therefore:

$T2 \rightarrow \text{WAIT for } T1$

We now have:

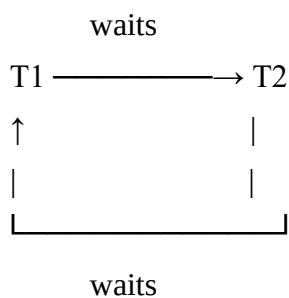
$T1$ waits for $T2$

$T2$ waits for $T1$

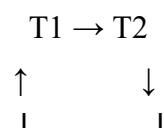
This creates a deadlock.

Draw the Wait-For Graph

The wait-for graph is:



Or simply:



There is a cycle:

$T1 \rightarrow T2 \rightarrow T1$

Therefore, a deadlock exists.

Identify the Deadlock

The deadlock is:

$T1$ holds A and waits for B

$T2$ holds B and waits for A

Transaction	Holds	Waiting For
$T1$	X-lock(A)	X-lock(B)
$T2$	X-lock(B)	X-lock(A)

Resolve the Deadlock

One transaction must be aborted/rolled back.

For example, abort T2.

T2 → ABORT

↓

Release X-lock(B)

↓

T1 obtains X-lock(B)

↓

T1 completes

↓

COMMIT T1

Restart T2

After T1 finishes:

T1:

X-lock(A)

X-lock(B)

Read/Write A

Read/Write B

COMMIT

Unlock A

Unlock B

Now T2 can restart:

T2:

X-lock(B)

X-lock(A)

Read/Write B

Read/Write A

COMMIT

Unlock B

Unlock A

The deadlock is removed.

Correct 2PL Execution

The corrected execution can be:

T1:

X-lock(A)

X-lock(B)

Read(A)
Write(A)
Read(B)
Write(B)
COMMIT
Unlock(A)
Unlock(B)

T2:
X-lock(B)
X-lock(A)
Read(B)
Write(B)
Read(A)
Write(A)
COMMIT
Unlock(B)
Unlock(A)

How to Avoid the Deadlock

A simple way to prevent this particular deadlock is to make all transactions request locks in the same order.

For example:

Always lock A first

Then lock B

So both transactions follow:

X-lock(A)

↓

X-lock(B)

↓

Perform operations

↓

Commit

↓

Release locks

Then T2 waits for T1 to release A instead of creating a circular wait.

Final Answer

T1 holds A → waits for B

T2 holds B → waits for A

↓

Deadlock

↓

Wait-for graph

T1 → T2

T2 → T1

↓

Cycle

↓

Abort T2

↓

Release B

↓

T1 completes

↓

Restart T2

Result

Basic Two-Phase Locking was successfully demonstrated. A deadlock occurred because T1 and T2 were waiting for each other's locks. The deadlock was identified using a wait-for graph and resolved by aborting T2, releasing its locks, allowing T1 to complete, and then restarting T2.

6. Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.

Ans:

Given Transactions

Consider three transactions with the following timestamps:

Transaction	Timestamp	Age
T1	10	Older
T2	20	Younger
T3	30	Youngest

Rule

The transaction with the smaller timestamp is older.

Therefore:

T1 (10) → Oldest

T2 (20) → Younger

T3 (30) → Youngest

Given Lock Requests

Assume the following situation:

T1 holds X-lock(A)

T2 requests X-lock(A)

T3 holds X-lock(B)

T1 requests X-lock(B)

So:

Transaction	Holds	Requests
T1	X-lock(A)	X-lock(B)
T2	—	X-lock(A)
T3	X-lock(B)	—

Apply Wait-Die Scheme

Wait-Die Rules

In Wait-Die:

- Older transaction requests a lock held by younger → WAIT
- Younger transaction requests a lock held by older → ABORT (DIE)

Case 1: T2 requests A

A is held by T1.

T2 timestamp = 20

T1 timestamp = 10

T1 is older than T2.

Therefore:

T2 (younger)

↓

Requests lock held by

↓

T1 (older)

↓
T2 ABORTS

So:

T2 → DIE/ABORT

Case 2: T1 requests B

B is held by T3.

T1 timestamp = 10

T3 timestamp = 30

T1 is older than T3.

Therefore:

T1 (older)

↓
Requests lock held by

↓
T3 (younger)

↓
T1 WAITS

So:

T1 → WAIT

Wait-Die Result

Request	Requesting Transaction	Lock Holder	Result
T2 requests A	T2 (younger)	T1 (older)	T2 aborts
T1 requests B	T1 (older)	T3 (younger)	T1 waits

Therefore:

Wait-Die:

Older → WAIT

Younger → DIE/ABORT

Apply Wound-Wait Scheme

Wound-Wait Rules

In Wound-Wait:

- Older transaction requests a lock held by younger → Younger is ABORTED
- Younger transaction requests a lock held by older → WAIT

Case 1: T2 requests A

A is held by T1.

T2 = 20 → Younger

T1 = 10 → Older

T2 is younger and requests a lock held by older T1.

Therefore:

T2 (younger)

↓

Requests A held by

↓

T1 (older)

↓

T2 WAITS

So:

T2 → WAIT

T1 requests B

B is held by T3.

T1 = 10 → Older

T3 = 30 → Younger

T1 is older and requests a lock held by younger T3.

Therefore:

T1 (older)

↓

Requests B held by

↓

T3 (younger)

↓

T3 ABORTS

So:

T3 → WOUNDED/ABORTED

Wound-Wait Result

Request	Requesting Transaction	Lock Holder	Result
T2 requests A	T2 (younger)	T1 (older)	T2 waits
T1 requests B	T1 (older)	T3 (younger)	T3 aborts

Therefore:

Wound-Wait:

Older → WOUND/ABORT Younger

Younger → WAIT

Compare Both Schemes

Situation	Wait-Die	Wound-Wait
Older requests younger's lock	Older waits	Younger aborts
Younger requests older's lock	Younger aborts	Younger waits
T2 → T1	T2 aborts	T2 waits
T1 → T3	T1 waits	T3 aborts

Easy Memory Trick**Wait-Die**

OLDER → WAIT

YOUNGER → DIE

Wound-Wait

OLDER → WOUND

YOUNGER → WAIT

Final Result**Wait-Die**

T2 requests A held by T1

↓

T2 is younger

↓

T2 ABORTS

T1 requests B held by T3

↓

T1 is older

↓

T1 WAITS

Wound-Wait

T2 requests A held by T1

↓

T2 is younger



T2 WAITS

T1 requests B held by T3



T1 is older



T3 ABORTS

Conclusion

Wait-Die and Wound-Wait prevent deadlocks by using transaction timestamps. Wait-Die allows older transactions to wait and aborts younger transactions, whereas Wound-Wait allows younger transactions to wait and aborts younger transactions when an older transaction requests their lock.

7. Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.

Ans:

Understand Immediate Update

In Immediate Update, changes made by a transaction can be written to the database **before the transaction commits**.

Therefore, after a crash:

- **Committed transactions → REDO**
- **Uncommitted transactions → UNDO**

So:

Immediate Update = UNDO + REDO

Given Transaction Log

Consider the following transaction log:

Log No.	Log Record
1	<START T1>
2	<T1, A, 100, 150>
3	<START T2>
4	<T2, B, 200, 250>
5	<COMMIT T1>
6	<CHECKPOINT>
7	<START T3>
8	<T3, C, 300, 350>

9	<COMMIT T2>
10	SYSTEM CRASH

The format:

<T1, A, 100, 150>

means:

Transaction = T1

Data item = A

Old value = 100

New value = 150

Identify the Checkpoint

The checkpoint occurs at:

<CHECKPOINT>

This tells the recovery system that earlier committed transactions may already have been safely processed.

The checkpoint helps reduce the amount of log that needs to be examined.

Identify Transaction Status at Failure

At the time of the crash:

T1

T1 → COMMITTED

because:

<COMMIT T1>

appears in the log.

T2

T2 → COMMITTED

because:

<COMMIT T2>

appears before the crash.

T3

T3 → UNCOMMITTED

because there is no:

<COMMIT T3>

before the crash.

Therefore:

Transaction	Status
T1	Committed

T2	Committed
T3	Uncommitted

Identify Transactions for REDO

In Immediate Update:

Committed $\rightarrow$ REDO

Therefore:

T1 $\rightarrow$ REDO

T2 $\rightarrow$ REDO

REDO T1

Log record:

$\langle T1, A, 100, 150 \rangle$

T1 committed.

Therefore, apply the new value:

$A = 150$

So:

Old value = 100

New value = 150

REDO operation

$A: 100 \rightarrow 150$

REDO T2

Log record:

$\langle T2, B, 200, 250 \rangle$

T2 committed.

Therefore:

$B = 250$

REDO operation

$B: 200 \rightarrow 250$

Identify Transaction for UNDO

In Immediate Update:

Uncommitted $\rightarrow$ UNDO

T3 did not commit.

Therefore:

$T3 \rightarrow$ UNDO

UNDO T3

Log record:

<T3, C, 300, 350>

The old value is:

300

The new value is:

350

Since T3 did not commit, restore the old value:

C = 300

Therefore:

C: 350 → 300

Recovery Table

Transaction	Status	Recovery Action	Final Value
T1	Committed	REDO	A = 150
T2	Committed	REDO	B = 250
T3	Uncommitted	UNDO	C = 300

Final Database State

After recovery:

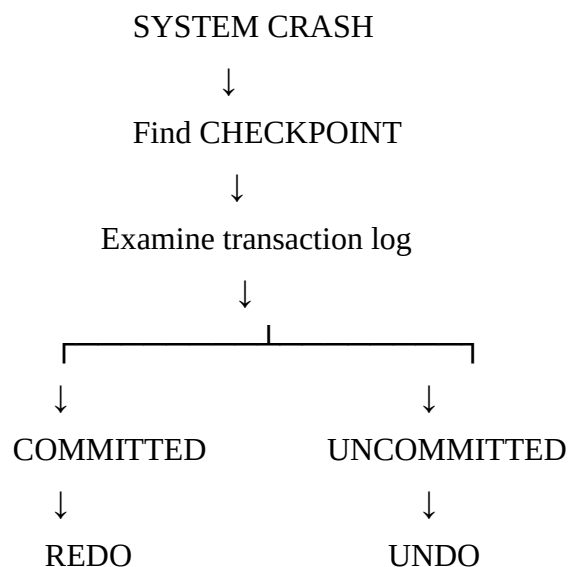
A = 150

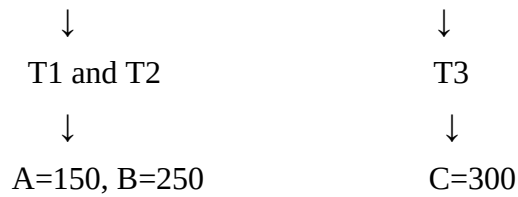
B = 250

C = 300

The effects of committed transactions are preserved, while the effect of the uncommitted transaction is removed.

Recovery Flow





Conclusion

Using the Immediate Update recovery protocol, T1 and T2 are REDONE because they committed before the system failure. T3 is UNDONE because it was uncommitted at the time of failure. Therefore, the final recovered database contains A = 150, B = 250, and C = 300.

8. Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.

Ans:

Understand Deferred Update

In Deferred Update, changes made by a transaction are not written to the database until the transaction commits.

Therefore, after a system crash:

- Committed transactions → REDO
- Uncommitted transactions → Ignore
- UNDO is not required

Given Transaction Log

Consider the following log:

Log No.	Log Record
1	<START T1>
2	<T1, A, 100, 150>
3	<START T2>
4	<T2, B, 200, 250>
5	<COMMIT T1>
6	<START T3>
7	<T3, C, 300, 350>
8	<COMMIT T2>
9	<START T4>
10	<T4, D, 400, 450>
11	SYSTEM CRASH

Identify Transaction Status

Look for a COMMIT record before the crash.

T1

<COMMIT T1>

Therefore:

T1 = Committed

T2

<COMMIT T2>

Therefore:

T2 = Committed

T3

There is no:

<COMMIT T3>

Therefore:

T3 = Uncommitted

T4

There is no:

<COMMIT T4>

Therefore:

T4 = Uncommitted

Prepare the Transaction Status Table

Transaction	Status	Recovery Action
T1	Committed	REDO
T2	Committed	REDO
T3	Uncommitted	Ignore
T4	Uncommitted	Ignore

REDO T1

Log record:

<T1, A, 100, 150>

T1 committed.

Therefore, apply the new value:

A = 150

So:

A: 100 → 150

REDO T2

Log record:

<T2, B, 200, 250>

T2 committed.

Therefore:

B = 250

So:

B: 200 → 250

Ignore T3

Log record:

<T3, C, 300, 350>

T3 did not commit.

Under Deferred Update, its changes were not written to the database.

Therefore:

T3 → IGNORE

No UNDO operation is performed.

Ignore T4

Log record:

<T4, D, 400, 450>

T4 also did not commit.

Therefore:

T4 → IGNORE

No UNDO operation is required.

Recovery Table

Transaction	Operation	Status	Recovery
T1	A: 100 → 150	Committed	REDO
T2	B: 200 → 250	Committed	REDO
T3	C: 300 → 350	Uncommitted	Ignore
T4	D: 400 → 450	Uncommitted	Ignore

Final Database State

After recovery:

A = 150

B = 250

T3 and T4 changes are not present because they were uncommitted.

Therefore:

C = 300

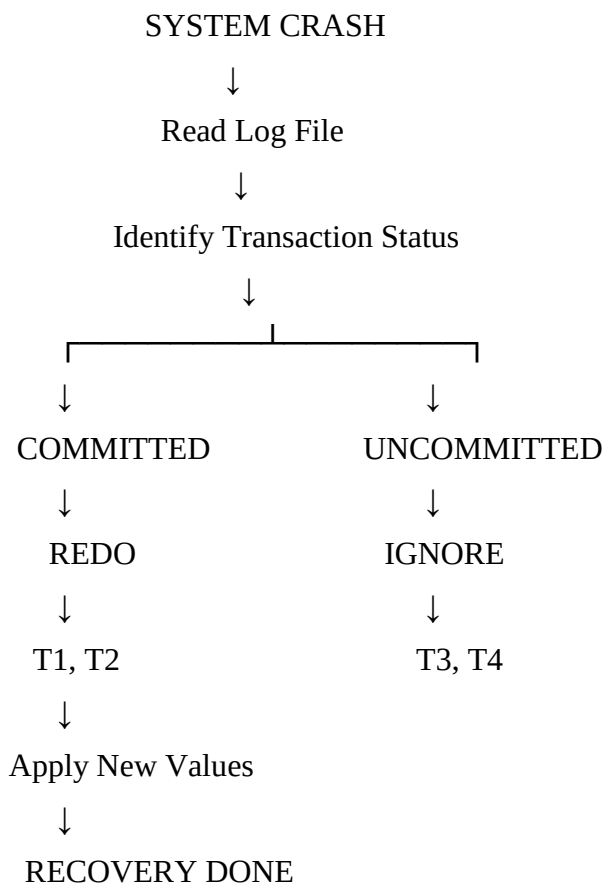
D = 400

if their original values were 300 and 400.

Final State

Data Item	Final Value
A	150
B	250
C	300
D	400

Recovery Flow



Conclusion

Using Deferred Update recovery, T1 and T2 are committed transactions, so their updates are REDONE. T3 and T4 are uncommitted transactions, so their updates are ignored. No UNDO operation is required because deferred update writes changes to the database only after a transaction commits.

9. Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.

Ans:

Define Transaction T1

T1 transfers ₹2,000 from Account A to Account B.

The transaction is:

T1:

Lock A

Lock B

↓

Read A

$A = A - 2000$

Write A

↓

Read B

$B = B + 2000$

Write B

↓

COMMIT

↓

Unlock A and B

```
mysql> CREATE DATABASE banking_db;
Query OK, 1 row affected (0.06 sec)

mysql> USE banking_db;
Database changed
mysql> CREATE TABLE account ( account_id INT PRIMARY KEY, account_name VARCHAR(50), balance DECIMAL(10,2) );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO account VALUES (101, 'Account A', 10000.00), (102, 'Account B', 5000.00);
Query OK, 2 rows affected (0.05 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM account;
+-----+-----+-----+
| account_id | account_name | balance |
+-----+-----+-----+
| 101 | Account A | 10000.00 |
| 102 | Account B | 5000.00 |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

Implement T1 in MySQL

Open MySQL Session 1.

Execute:

USE banking_db;

START TRANSACTION;

```
SELECT *  
FROM account  
WHERE account_id IN (101,102)  
FOR UPDATE;  
FOR UPDATE obtains exclusive row locks on the selected accounts.
```

Now transfer the money:

```
UPDATE account  
SET balance = balance - 2000  
WHERE account_id = 101;
```

Then:

```
UPDATE account  
SET balance = balance + 2000  
WHERE account_id = 102;
```

At this point T1 has changed the balances but **has not committed**.

Define Transaction T2

T2 wants to read the balances while T1 is transferring money.

T2 should use shared/read locks if we want to explicitly demonstrate strict locking.

Open **MySQL Session 2** and execute:

```
USE banking_db;
```

```
START TRANSACTION;
```

```
SELECT *  
FROM account  
WHERE account_id IN (101,102)  
FOR SHARE;
```

Because T1 is still holding exclusive locks, T2 has to **wait** until T1 commits or rolls back.

T1 holds X-lock

↓

T2 requests S-lock

↓

T2 waits

Commit T1

Go back to **MySQL Session 1**.

Execute:

COMMIT;

Now T1 releases its locks.

T1

↓

COMMIT

↓

Release locks

↓

T2 can continue

T2 Reads the Updated Balance

T2 can now obtain its shared locks and read the balances.

SELECT *

FROM account

WHERE account_id IN (101,102);

Expected result:

Account ID	Balance
101	₹8,000
102	₹7,000

Then finish T2:

COMMIT;

What Happens Without Strict 2PL?

Suppose T2 reads while T1 has changed Account A but has not yet changed Account B:

T1: A = A - 2000

↓

T2: Reads A = 8000

↓

T2: Reads B = 5000

↓

T1: B = B + 2000

T2 sees:

A = 8000

B = 5000

Total = ₹13,000.

But the correct total before the transfer was:

10000 + 5000 = ₹15000

Therefore T2 has read an **inconsistent state**.

Strict 2PL prevents this.

Strict 2PL Mechanism

The mechanism is:

T1

X-lock(A)

X-lock(B)

↓

Update A

Update B

↓

COMMIT

↓

Unlock A

Unlock B

T2

Request S-lock(A)

Request S-lock(B)

↓

WAIT

↓

T1 COMMIT

↓

Acquire locks

↓

Read balances

↓

COMMIT

Final State

Initial:

Account A = ₹10,000

Account B = ₹5,000

T1 transfers ₹2,000:

Account A = ₹8,000

Account B = ₹7,000

T2 reads only after T1 commits:

Account A = ₹8,000

Account B = ₹7,000

Total remains:

₹8,000 + ₹7,000 = ₹15,000

So the banking system remains consistent.

Result

A Strict 2PL concurrency control mechanism was successfully designed for the banking system. T1 obtains exclusive locks on both accounts while transferring funds and holds them until commit. T2 waits until T1 commits before reading the balances. Thus, dirty reads and inconsistent intermediate balances are prevented, ensuring database consistency and reliability.

10. Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.

Ans:

Objective:

To optimize a query by reducing unnecessary data before performing joins.

Step 1: Start with the Query

Input: SQL Query

Step 2: Convert Query

Convert SQL query into a query tree.

Step 3: Push Down Selection

Move selection conditions closer to the base tables.

Example:

Before:

JOIN → SELECT

After:

SELECT → JOIN

This reduces the number of rows before joining.

Step 4: Project Early

Keep only the required columns from each table.

Example:

STUDENT → Keep Student_ID, Name, Dept_ID

This reduces the amount of data processed.

Step 5: Order Joins

Estimate the size of intermediate results.

Perform the join producing the smaller result first.

Step 6: Generate Optimized Query Tree

Selection



Projection



Smallest Join



Next Join



Final Result

Pseudocode

Algorithm Optimize_Query(Query)

1. Parse Query
 2. Create Query Tree
 3. For each selection condition:
 - Push selection toward base table
 4. For each table:
 - Keep only required attributes
 - Push projection toward base table
 5. Estimate intermediate result sizes
 6. Order joins:
 - Perform smaller-result joins first
 7. Generate optimized query tree
 8. Return optimized query
- End Algorithm

Result

Heuristic query optimization reduces intermediate data, disk I/O, memory usage, and query execution time by pushing selections and projections early and performing smaller joins first.

